Sistema Inteligente de Detección de Spam (Spambase) con Asistente Conversacional y API RESTful

1st Emary Galarza Quilca
Facultad de Ingeniería de Sistemas
UNCP
Huancayo, Perú

2nd Miguel Angel Huiza Vazquez
Facultad de Ingeniería de Sistemas
UNCP
Huancayo, Perú

3rd Ares Gabriel Castillo Ayarza
Facultad de Ingeniería de Sistemas
UNCP
Huancayo, Perú

4th Italo Osmar Grijalva Martinez
Facultad de Ingeniería de Sistemas
UNCP
Huancayo, Perú

***Resumen*—En el presente trabajo se diseñó, implementó y evaluó un sistema integral de clasificación e inferencia en tiempo real para detectar correos electrónicos no deseados (Spam). Partiendo del dataset público UCI Spambase (4,601 registros y 57 características cuantitativas), se estructuró un pipeline libre de fuga de datos (*Data Leakage*), comparando modelos lineales y Redes Neuronales Multicapa (MLP). El modelo promovido a producción (MLP v1) alcanzó un F1-Score de 0.9224 en el conjunto de prueba aislado. La solución se desplegó mediante una API RESTful en FastAPI (`/predecir`, `/salud`) y un Asistente Conversacional en Streamlit con traducción multilingüe al vuelo, explicabilidad de señales y estricto cuidado ético (procesamiento volátil en RAM). La calidad del software fue validada al 100 % mediante una suite de pruebas automáticas con `pytest`.**

***Index Terms*—Spambase, Redes Neuronales MLP, FastAPI, Streamlit, Cuidado Ético, Pruebas Automáticas.**

I. Introducción y Planteamiento (Etapa E1)

Las bandejas de entrada se encuentran fuertemente saturadas por mensajes promocionales y estafas (Spam), generando pérdida de tiempo y vulnerabilidad a estafas de *phishing*. El usuario objetivo (estudiantes, profesionales y usuarios comunes) corre el riesgo de traspapelar notificaciones críticas (ofertas laborales, facturas, requerimientos legales) al confundirlas con correo basura.

I-A. Momento de Predicción y Datos Disponibles

La predicción ocurre en tiempo real en la interfaz del chatbot, en el preciso instante en que el usuario copia y pega el texto de un correo sospechoso, antes de interactuar con él (hacer clic en un enlace o descargar adjuntos). En ese momento, el sistema extrae un vector de 57 características estadísticas estandarizadas derivado del texto.

I-B. Costo de los Errores y Criterio de Métrica

- **Falso Positivo (FP):** Clasificar por error un correo legítimo como Spam. Es el error más costoso, ya que oculta información vital para el usuario, derivando en problemas laborales o legales.
- **Falso Negativo (FN):** Permitir que un correo Spam llegue a la bandeja principal (molestia menor).
- **Métrica Elegida:** F1-Score, Precisión y curva PR-AUC para proteger los correos legítimos frente al desbalance.

I-C. Riesgos Éticos y Medidas Concretas

Se previene la violación de privacidad procesando la información de manera exclusiva en **memoria volátil (RAM)** sin almacenamiento persistente (ver Fig. 8), incluyendo derivación simulada a soporte humano (ver Fig. 9).

II. Datos y Exploración sin Fuga (Etapa E2)

Se trabajó con el dataset UCI Spambase (ID 94), conformado por 4,601 registros y 57 variables predictoras continuas más la clase binaria.

- **Auditoría de Calidad:** 0 nulos nativos, manejo de 391 filas duplicadas mediante particionamiento por huella para evitar contaminación, y conservación justificada de *outliers* en frecuencias de caracteres.
- **Partición Estricta:** Entrenamiento (60

III. Modelado y Comparación (Etapa E3)

Se entrenaron modelos de complejidad creciente sobre el conjunto de entrenamiento procesado (ver entrenamiento en Fig. 1):

- **Dummy Baseline:** F1 en Validación = 0.0000.
- **Regresión Logística:** F1 en Validación = 0.8380.
- **Red Neuronal MLP v1 (1 capa de 50 neuronas):** F1 en Validación = **0.9183**.
- **Red Neuronal MLP v2 (2 capas con Dropout y Early Stopping):** F1 en Validación = 0.9179.

IV. Evaluación y Versionado (Etapa E4)

La selección del modelo ganador se basó exclusivamente en validación (ver Fig. 2). El conjunto de prueba aislado (921 muestras) arrojó los siguientes resultados definitivos:

Cuadro I
COMPARATIVA DE RENDIMIENTO EN CONJUNTO DE PRUEBA

Modelo	F1 (Val)	F1 (Prueba)	Precisión	Recall
Dummy Baseline	0.0000	0.0000	0.0000	0.0000
Regresión Logística	0.8380	0.8819	0.9021	0.8625
MLP v1 (Ganador)	**0.9183**	**0.9224**	**0.9380**	**0.9074**
MLP v2	0.9179	0.9154	0.9315	0.9000

Se generaron fichas técnicas JSON por versión y se promovió a producción el archivo `modelo_produccion.joblib`.

V. DESPLIEGUE Y API RESTFUL (ETAPA E5)

Implementada en FastAPI (`src/api.py`), con documentación interactiva (ver Fig. 3):

- **Warm-up:** Carga en memoria RAM al arrancar el servidor.
- **Endpoints:** `GET /salud` (ver Fig. 4) y `POST /predecir` (ver Fig. 5).
- **Validación Pydantic:** Retorna HTTP 400 Bad Request ante dimensiones incorrectas (ver Fig. 6).

VI. ASISTENTE CONVERSACIONAL Y CHATBOT (ETAPA E6)

Desarrollado en Streamlit (`src/chatbot.py`):

- **Pipeline Multilingüe:** Traducción automática al vuelo al inglés mediante *deep-translator*.
- **Diagnóstico y Explicación:** Detalla señales de alerta de spam (ver Fig. 7).
- **Flujo y Fallbacks:** Incluye caminos feliz, alternativo, de error y derivación a soporte humano (ver Fig. 10 y Fig. 9).

VII. CALIDAD Y REPRODUCIBILIDAD (ETAPA E7)

Suite de pruebas automáticas ejecutada al 100 % en verde mediante `pytest` (ver Fig. 11), apoyada por el archivo de configuración global `pytest.ini`, gestión de dependencias y la guía del `README.md`.

VIII. CONCLUSIONES Y TRABAJO FUTURO (ETAPA E8)

El sistema cumple con los requerimientos de la rúbrica. Como trabajo futuro, se contempla migrar la heurística a un modelo de lenguaje basado en Transformers adaptado de forma directa al español.

APORTES Y CONTRIBUCIONES INDIVIDUALES

- **Emary Galarza Quilca (E1 y E2):** Planteamiento del problema, costos de error, auditoría de calidad de datos y partición estricta sin fuga de datos.
- **Miguel Angel Huiza Vazquez (E3):** Línea base, desarrollo de redes neuronales MLP con regularización y automatización del entrenamiento.
- **Ares Gabriel Castillo Ayarza (E4 y E5):** Evaluación de métricas, versionado JSON, desarrollo de la API en FastAPI y esquemas de validación Pydantic.
- **Italo Osmar Grijalva Martinez (E6, E7 y E8):** Chatbot en Streamlit con traducción al vuelo, diagramas, suite de pruebas en `pytest` y consolidación de documentación.

DECLARACIÓN SOBRE EL USO DE IA GENERATIVA

Se empleó IA generativa como co-piloto para optimizar fragmentos de código repetitivo, plantillas de Pydantic y soporte de diagramación en Mermaid. Se subraya que el análisis de datos, las decisiones de preprocesamiento, el diseño en el cuaderno Jupyter y la validación técnica fueron íntegramente desarrollados de forma intelectual por el equipo.

ANEXOS: EVIDENCIAS DE EJECUCIÓN

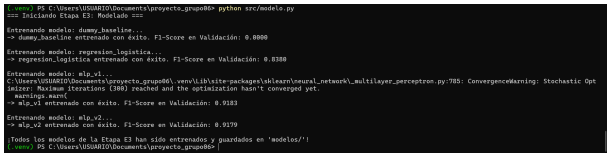


Figura 1. Entrenamiento de modelos de la Etapa E3 y F1-Score en validación.

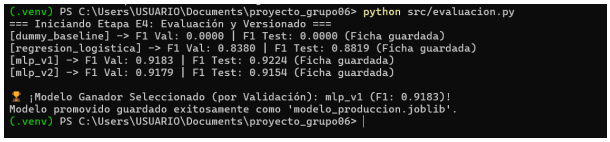


Figura 2. Selección del modelo ganador (MLP v1) y promoción a producción.



Figura 3. Documentación interactiva Swagger UI de la API RESTful.

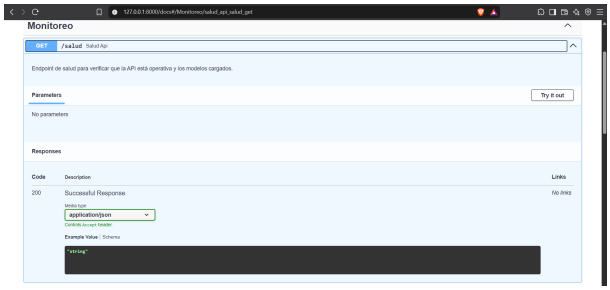


Figura 4. Verificación operativa de la API en la ruta `/salud`.

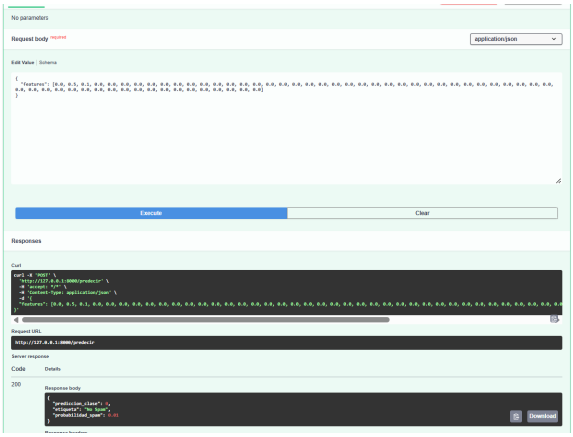


Figura 5. Prueba de inferencia exitosa en la API (`POST /predecir`).

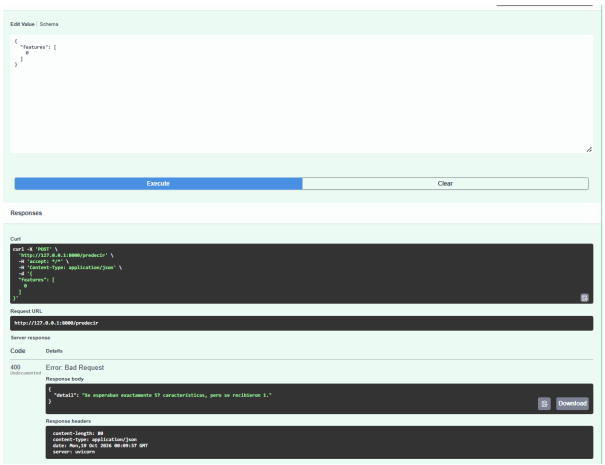


Figura 6. Manejo de errores HTTP 400 ante dimensiones incorrectas.

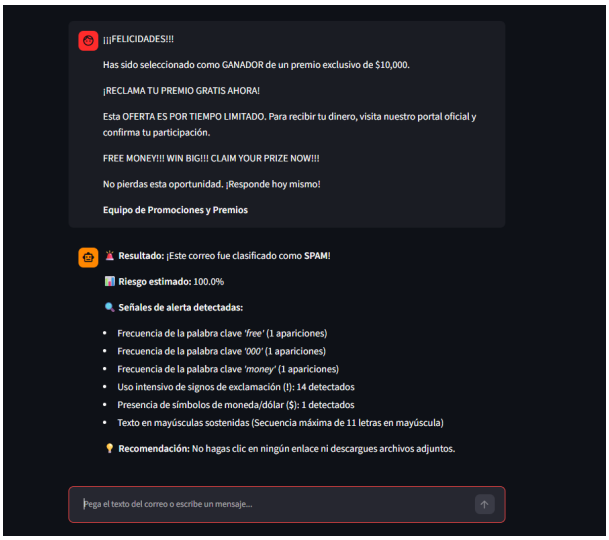


Figura 7. Interfaz del Chatbot en Streamlit con análisis y recomendaciones.



Figura 8. Aviso de Privacidad y Cuidado Ético en el chatbot (memoria RAM).

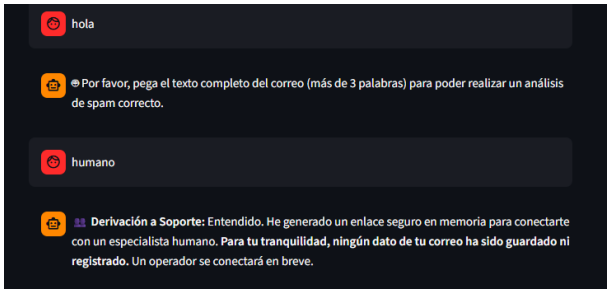


Figura 9. Respuesta de Fallback y derivación simulada a soporte humano.

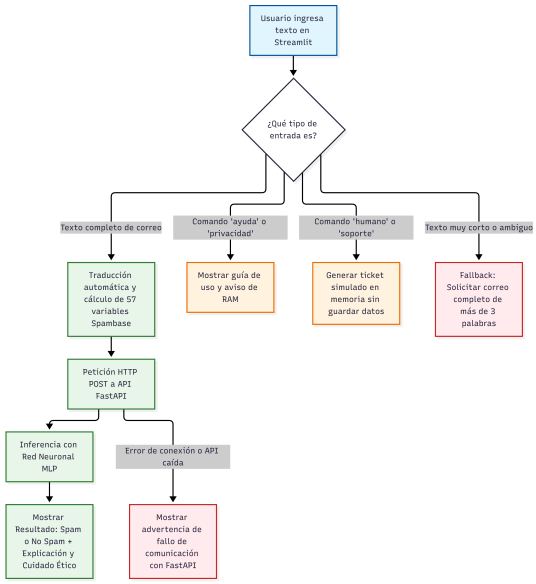


Figura 10. Diagrama de flujo del chatbot (caminos feliz, alternativo y error).

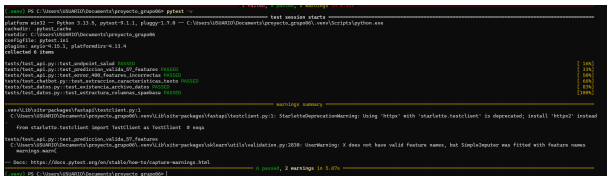


Figura 11. Ejecución exitosa de la suite de pruebas automáticas con `pytest`.